

tenXengage

Redemption Store Integration

Engineering Architecture Analysis

HIGHLY CONFIDENTIAL | INTERNAL USE ONLY

Document Version: 1.0 | April 2026

Stack: Java 21 + Spring Boot 3.2.5 + PostgreSQL 16 + React 18 + TypeScript

Prepared by: Senior Engineering Architect

1. Executive Summary

The Redemption Store Integration closes the earn-to-redeem loop on the tenXengage platform by delivering a unified, currency-aware reward redemption experience backed by two purpose-built vendor integrations: **XTRM** for global cash payouts (ACH, PayPal, Venmo, 180+ country wire) and **Xoxoday** for non-cash rewards (gift cards, merchandise, travel, vendor events). tenXengage retains full ledger ownership — both vendors operate purely as fulfillment engines, invisible to end users — which is the architectural posture that enables the platform's compliance delegation model and vendor-agnostic UX.

The core architectural differentiator is the **ledger-first, two-step balance safety model**: every balance mutation produces an immutable LedgerEntry before wallet totals are updated, and the reserve-on-submission / debit-on-confirmation pattern eliminates double-spend without blocking the user during async vendor fulfillment. This design is independent of vendor uptime — wallet state is always derivable from the ledger even if a vendor webhook is delayed or lost.

The **dual-wallet model** (individual seller + pooled company wallet, scoped per tenant) directly maps to enterprise channel partner structures where both personal incentives and team-level program rewards must coexist. The multi-tenant isolation layer (client_id, Hibernate @Filter, TenantValidator) is inherited from the platform and must be applied consistently across all six new entities.

Both source documents are substantially aligned. The Prompt Engineering Document covers the full service, entity, and frontend layer in implementation-ready prompts with correct platform conventions. However, **two material gaps exist**: (1) the Kafka event publishing layer (15 event types defined in the BRD) has zero coverage in the Prompt Doc, and (2) the admin-facing UI layer (Client Admin approval queue, catalog configurator, tenant reporting; Platform Admin catalog manager) is defined in the BRD but not prompted. These gaps must be addressed before Sprint 1 begins.

This quadrant is assessed as **READY WITH CONDITIONS**. The data model, service layer, vendor adapters, webhook handlers, returns engine, and partner-facing frontend are all prompt-ready and architecturally sound. The conditions are: Kafka event publishing spec, admin frontend prompt coverage, webhook dead-letter strategy decision, and EXPIRY_DEBIT enum removal from the Sprint 1 migration.

2. BRD vs. Prompt Engineering Doc — Alignment Analysis

Key: GAP = missing coverage | NOTE = minor divergence | EXPIRY = enum scope issue

Capability	BRD Coverage	Prompt Doc Coverage	Gap / Note
Reward Wallet Model (individual + company, per-currency balances)	Full — Section 8.1, 11.2. Owner types, balance fields,	Full — Prompts 3.1 + 4.1. Entity, migration, WalletService with	None. Prompt 3.1 refinement correctly adds @Version for optimistic locking — matches BRD NFR.

Capability	BRD Coverage	Prompt Doc Coverage	Gap / Note
	reserve/available split defined.	all mutation methods.	
Ledger Engine (immutable LedgerEntry, CREDIT/DEBIT/RESERVE/RELEASE/RETURN_CREDIT)	Full — Section 8.2, 11.3. Entry types, source types, balance derivability.	Full — Prompt 3.2 + 4.1. Immutability enforced, amount > 0 CHECK constraint, InOrder test verification.	Prompt Doc adds EXPIRY_DEBIT to LedgerEntryType enum. BRD Section 5.3 explicitly defers balance expiration to Phase 2. This enum value must NOT be generated in Sprint 1 migrations.
Redemption Catalog (global + tenant config, currency-aware filtering)	Full — Sections 8.3, 11.4, 11.5.	Full — Prompts 3.3, 4.2. Geographic + currency filters, shortfall calc, sync job.	None.
Redemption Flow + Vendor Routing (XTRM/Xoxoday, 3 processing modes)	Full — Section 8.4, 11.6.	Full — Prompts 4.3, 4.4. Orchestration service with mode handling, both adapters.	Prompt Doc 4.3 references routeToVendor() but does not define the retry behavior inline, deferred to adapters via @Retryable. Ensure retry config values (maxAttempts, backoff) are defined in application.yml, hardcoded.
Webhook Handling (XTRM + Xoxoday, HMAC-SHA256, idempotency)	Section 12.5, 13, 21.	Full — Prompt 4.5. Both handlers with HMAC-first, idempotency check, 200-always pattern.	None on handler logic. BRD mentions dead-letter queue for unprocessable webhooks (Section 14, 21) but Prompt Doc only says "dead-letter logging". No Kafka DLQ or DB dead-letter table is specified. This needs decision before Sprint 1.
Returns Flow (non-cash only, 5-state lifecycle, vendor gate)	Full — Section 8.5, 11.7, 12.4.	Full — Prompt 4.6. All 5 return states, CASH rejection, vendor gate enforced.	None.
REST API Surface (wallet, catalog, redemption, returns, webhook controllers)	Full — Section 12.1–12.5.	Full — Prompt 4.7. All 21 endpoints mapped with correct permissions.	BRD does not define a GET /api/v1/redemption/catalog/ad endpoint for Platform Admin catalog management. Prompt Doc does not generate a PlatformAdminCatalogController. Gap: platform admin catalog CRUD is referenced in Section (frontend) but has no backend endpoint spec.
Kafka Event Architecture (15 event types)	Full — Section 13. All events listed.	Not addressed. Prompt Doc generates no Kafka producers, no event classes, no topic definitions.	SIGNIFICANT GAP. The Prompt Doc has no section for event publishing. This must be addressed before Sprint 1 or the analytics notification, and PAS pipeline will have no integration point.

Capability	BRD Coverage	Prompt Doc Coverage	Gap / Note
			Recommend adding a RedemptionEventPublisher service prompt.
Notifications (11 event-to-recipient mappings)	Full — Section 19. All notification events mapped.	Not addressed. No notification service integration in any prompt.	GAP. Notification publishing must be triggered from webhook handlers and service methods. No prompt generated for this integration. Needs explicit prompt or inline addition to existing service prompts.
Audit Logging (@Audited, AuditAction, AuditResourceType)	Full — Section 20.	Partial — Prompt 4.7 adds @Audited to controllers. No prompt generates SYSTEM-actor audit entries (wallet credited by webhook).	SYSTEM-actor audit entries (webhook processing, wallet credit) are not covered by controller-level @Audited. Need explicit AuditService calls inside XtrmWebhookService and XoxodayWebhookService.
Frontend: Wallet Dashboard, Catalog, Transaction History, Balance Widget	Section 24.	Full — Prompts 5.2–5.5. All four areas covered with detailed specs.	None on component scope. However, Client Admin frontend areas (Approval Queue, Catalog Configurator, Tenant Reporting and Platform Admin (Global Catalog Manager) are described in BRD Section 24 but Prompt Doc only covers partner-facing views. Admin UIs need separate prompt coverage.
Permissions + Feature Flags (11 keys, 3 tiers)	Full — Section 10.	Full — Prompt 7.1. All 11 keys, feature flag, role mappings via INSERT...SELECT.	None.
Observability / MDC / Metrics	Section 14 (NFR)	Full — Prompt 7.2. MDC filter, log statements, 6 Micrometer metrics.	None on metric coverage. BRD mentions SLA breach monitor (Phase 2) — not in Prompt Doc. Correct.
Testing: Unit + Integration + E2E	Section 18 (acceptance criteria)	Full — Prompts 6.1, 6.2, 6.3. WalletService unit, webhook Testcontainers, Playwright E2E.	No test prompts for RedemptionOrchestrationService, ReturnsService, or catalog service. Only WalletService and webhook handlers are covered. Gap for Sprint 1 if test coverage gates are enforced.
PAS / Commercial Intent Signal Integration	Phase 2 deferred — Section 16, Track C.	Not addressed. Correctly deferred.	None. Correctly aligned.
Batch Processing Scheduler	Phase 2 deferred —	Not addressed.	None. Correctly deferred.

Capability	BRD Coverage	Prompt Doc Coverage	Gap / Note
	Section 16, Track C.		

3. Architecture Decisions to Confirm Before Sprint 1

Decision 1: Webhook Dead-Letter Strategy

Decision: Define the failure path for unprocessable vendor webhooks — DB dead-letter table, Kafka DLQ, or both.

Why it matters: The BRD mandates dead-letter handling (Section 14, 21). The Prompt Doc only says "dead-letter logging," which is insufficient. If a webhook cannot be processed (unparseable payload, missing transaction), the vendor has already executed the fulfillment — the balance is in a suspended state. Without a durable dead-letter store, these events are unrecoverable without vendor re-inquiry.

What breaks if deferred: Wallet inconsistencies on vendor-side errors become invisible. Customer support has no tooling to investigate failed webhook events. Compliance audit trail has holes.

Decision 2: Kafka Event Publishing — Topic Names, Schema Format, Producer Ownership

Decision: Define the 15 Kafka topic names (e.g., `tenx.redemption.completed`, `tenx.wallet.balance_updated`), payload schema (Avro vs. JSON), and which service class owns each publisher.

Why it matters: The entire downstream pipeline — notifications, analytics, PAS scoring (Phase 2) — depends on these events. The BRD defines all 15 event types (Section 13) but the Prompt Doc has no Kafka coverage whatsoever. No producer = no downstream integration possible from day 1.

What breaks if deferred: Notification service cannot trigger on redemption lifecycle. Analytics cannot track redemption rates. Other quadrant teams cannot listen to wallet events. Phase 2 PAS integration is blocked.

Decision 3: Optimistic vs. Pessimistic Locking on Wallet Balance

Decision: Confirm that `reserve()` uses `@Lock(PESSIMISTIC_WRITE)` on the wallet row SELECT, and document the decision. Prompt Doc Section 4.1 refinements suggest adding `@Version` for optimistic locking, but also describes pessimistic locking in the main `reserve()` spec. These are mutually exclusive approaches for the critical path.

Why it matters: Under high concurrency (partner submits two redemptions simultaneously), the wrong locking strategy allows double-spend — both submissions pass the `availableBalance >= amount` check before either commits.

What breaks if deferred: Race condition in production. Reserved balance can exceed available balance. Ledger becomes inconsistent. This is the highest-severity correctness risk in the feature.

Decision 4: Idempotency Key Storage for Vendor API Calls

Decision: Define how idempotency keys are stored and checked for XTRM and Xoxoday outbound calls. The Prompt Doc uses tx.id (UUID) as the externalId/referencId, which is correct for first submission. However, the retry path (@Retryable on adapters) will re-submit the same UUID — confirming that both XTRM and Xoxoday honor the externalId for deduplication must be verified before sprint.

Why it matters: If XTRM does not treat externalId as an idempotency key, retries create duplicate payouts. This is an AML and financial integrity issue.

What breaks if deferred: Duplicate cash payouts on vendor retry. Non-trivial to unwind once money has moved.

Decision 5: EXPIRY_DEBIT Enum Scope — Remove from V002 Migration

Decision: Remove EXPIRY_DEBIT from the LedgerEntry entryType enum in migration V002. BRD Section 5.3 explicitly defers balance expiration to Phase 2.

Why it matters: If EXPIRY_DEBIT ships in V002 and no Phase 1 code references it, it is dead schema. If a developer writes an accidental reference, it creates a Phase 2 dependency in Phase 1 code. If removed later, it requires a Flyway migration to alter an enum, which is PostgreSQL-version-sensitive.

What breaks if deferred: Schema drift between documented intent and actual DB. Potential compile-time reference to an enum value that has no Phase 1 behavior.

Decision 6: Catalog Sync Job Execution Model (Scheduled vs. On-Demand)

Decision: Determine whether syncXoxodayCatalog() runs on a @Scheduled cron, is triggered via a Platform Admin API endpoint, or both. The Prompt Doc says "@Scheduled or triggered manually" — this ambiguity must be resolved before implementation.

Why it matters: If catalog items drift between Xoxoday and the local DB (items deactivated on Xoxoday but still active locally), users can submit redemptions for non-existent items. The sync cadence determines drift risk.

What breaks if deferred: Redemption submissions routed to Xoxoday for retired items. Vendor returns 404 or error. Failed redemption, balance in reserved state, unhappy partner.

Decision 7: Platform Admin Catalog Backend Endpoints

Decision: Define and spec the missing Platform Admin backend endpoints for global catalog management (CRUD on RedemptionCatalogItem, XTRM/Xoxoday item mappings, vendor health dashboard). BRD Section 12 and 24 reference this surface; Prompt Doc generates no PlatformAdminCatalogController.

Why it matters: Without backend endpoints, the Platform Admin cannot manage the global catalog at launch. All catalog management requires direct DB access, which is operationally unacceptable.

What breaks if deferred: Go-live requires manual DB writes to manage catalog items. No audit trail on catalog changes. Immediate post-launch operational burden.

4. Agent Architecture Review

Important context: The tenXengage platform contains AI sub-agents in its Enablement quadrant (Readiness Gap Detection, Learning Path Recommendation, Deal-Context Training). The Redemption Store is NOT the Enablement quadrant — it is the Rewards & Redemption quadrant. Neither the BRD nor the Prompt Engineering Document for this feature defines or references AI sub-agents.

The Redemption Store BRD references the **Partner Activation Score (PAS)** as a Phase 2 downstream consumer of redemption events (Commercial Intent component, 18% weight). This is a signal-consumer relationship, not an agent architecture. The PAS pipeline is owned by a separate quadrant.

Assessment: Agent architecture review is NOT APPLICABLE to this feature as scoped. If this POD is responsible for the Enablement quadrant agents in addition to the Redemption Store, that is a scope definition error and must be clarified with the Program Manager immediately. This analysis is limited to the Redemption Store as documented.

5. Certification & Readiness Score Design

Assessment: The Partner Readiness Score (PRS) and CertificationStatus enum belong to the Enablement quadrant, not the Redemption Store. Neither the BRD nor the Prompt Engineering Document for this feature defines a PRS or CertificationStatus.

What this feature DOES define is the **Partner Activation Score (PAS) signal contribution** from redemption activity: BRD Section 13 and the Phase 2 release track state that redemption events should be routed into the ActivationSignal pipeline as a signal for the Commercial Intent component (18% PAS weight). This is deferred to Phase 2 and requires:

- The Redemption Store to publish wallet_balance_updated and redemption_completed events to Kafka (Sprint 1 prerequisite — see Gap in Section 2)
- The PAS scoring pipeline (separate team) to consume and weight these signals
- No CertificationStatus enum is defined or required in this feature

Action required: If this engineering analysis was requested in the context of an Enablement quadrant review, the wrong BRD and Prompt Doc have been loaded. Confirm scope with the PM.

6. Cross-Quadrant Dependencies

The Redemption Store is largely self-contained. Based on both documents, the following outbound and inbound interface dependencies exist:

Dependency	What Is Needed	Direction	Owning Team	Contract Defined?
Existing Notification Framework	NotificationCategory.REWARDS events must be publishable from webhook handlers and service methods for 11 event types (Section 19)	Outbound (Redemption Store calls)	Platform / Notification Team	Partial — category enum exists; trigger interface unclear
Kafka Event Bus	15 event types must be published for analytics, notifications, and PAS pipeline (Section 13)	Outbound (Redemption Store publishes)	Platform Infra / This POD	NOT DEFINED — topics, schema, producer unspecified
Existing Reward Earning System (Incentives/Engagement)	RewardWallet.credit() must be triggered when partners earn rewards from incentive programs, training completions, deal closures (existing TransactionType.REWARD events)	Inbound (Redemption Store consumes or is called)	Incentives / Engagement PODs	NOT CONFIRMED — does existing reward earning call WalletService.credit() or publish to Kafka?
PAS Scoring Pipeline (Phase 2)	Consumes redemption_completed and wallet_balance_updated events as Commercial Intent signals	Outbound (Redemption Store publishes, PAS consumes)	Enablement / PAS POD	Deferred — Phase 2. Kafka publish is Sprint 1 prerequisite.
Permission / RBAC System	@RequiresPermission AOP must resolve 11 new permission keys (V008 migration)	Inbound (Redemption Store uses)	Platform	Defined — reusing existing @RequiresPermission AOP
Audit Framework	@Audited annotation + AuditAction/AuditResourceType enums. SYSTEM-actor entries for webhook processing.	Inbound (Redemption Store uses)	Platform	Partial — controller-level handled; webhook SYSTEM audit not fully specified

Critical unresolved: How does the existing reward earning system (incentive program completions, deal closures) currently interact with the new RewardWallet entity? If the existing TransactionType.REWARD events are consumed via Kafka and this POD's WalletService.credit() is the subscriber, that integration point must be designed and contracted with the Incentives/Engagement PODs before Sprint 1. If earning is still handled by the existing transaction table and WalletService.credit() is only called by future Kafka consumers, the existing earning flow is unchanged and this is not a Sprint 1 concern. THIS IS AMBIGUOUS IN BOTH DOCUMENTS.

7. Sprint Readiness Assessment

Pre-Sprint Gate Item	Status	Impact if Unresolved
Contracts repo: entity enums (TransactionStatus reuse, new enums confirmed)	OPEN	Block Sprint 1 entity generation
Kafka topic names and event schema contracts defined	OPEN	Block event publishing implementation
XTRM + Xoxoday sandbox API credentials provisioned	OPEN	Block adapter development
Webhook signing secrets provisioned and stored in secrets manager	OPEN	Block webhook handler implementation
Cross-quadrant API contracts agreed (if any dependencies exist)	OPEN — Redemption Store appears self-contained; confirm with other PODs	Minor risk
Dead-letter strategy decision (DB table vs. Kafka DLQ)		Block webhook reliability implementation
Admin frontend scope confirmed in Sprint 1 plan		Risk of launching without admin tooling
RedemptionEventPublisher prompt/spec added to Prompt Doc		Block downstream integrations
EXPIRY_DEBIT removed from V002 migration	OPEN	Schema debt if merged as-is
Platform Admin catalog backend endpoints specified	OPEN	Gap in API surface coverage

Overall Readiness Rating: **READY WITH CONDITIONS**

Conditions (all must be resolved before first Sprint 1 story is pulled into active development):

1. Kafka event publishing spec added to Prompt Doc and topic names / schema agreed with platform team.
2. Webhook dead-letter strategy decided (DB table preferred for traceability) and implementation added to Prompt Doc.
3. Admin frontend prompt coverage added: Client Admin Approval Queue, Catalog Configurator, Tenant Reporting, and Platform Admin Global Catalog Manager.
4. EXPIRY_DEBIT removed from V002 Flyway migration.
5. XTRM and Xoxoday sandbox API credentials confirmed and accessible in the dev environment.
6. Reward earning integration point clarified: does existing earning flow need to call WalletService.credit() in this sprint, and if so, which team triggers it?
7. Platform Admin backend catalog endpoints specified and added to Prompt Doc Section 4 or a new Section 4.8.

8. Top Risks

Risk	Prob	Impact	Mitigation
Kafka event publishing not in Prompt Doc	H	H	Add RedemptionEventPublisher service prompt immediately. Define topic names, event schemas (Avro or JSON), and producer config before Sprint 1. Downstream notification and analytics teams are blocked without this.
Wallet concurrent update race condition (double-reserve)	H	H	Prompt Doc recommends pessimistic write lock in reserve(). Confirm @Lock(PESSIMISTIC_WRITE) is applied to the wallet row fetch in WalletService, not just @Transactional. Add integration test that fires concurrent reserve() calls against same wallet and asserts only one succeeds or both succeed without exceeding balance.
Vendor webhook dead-letter strategy undefined	M	H	BRD says DLQ; Prompt Doc says log. Decide: DB dead-letter table vs. Kafka DLQ vs. both. If a webhook cannot be processed (transaction not found, parse error), the vendor has already confirmed — balance may be in wrong state. Decision must be made and implemented in Sprint 1, not deferred.
EXPIRY_DEBIT enum in Sprint 1 migrations creates schema debt	M	M	Prompt 3.2 includes EXPIRY_DEBIT in the LedgerEntry migration, but Phase 2 defers expiry. Remove it from V002 migration. Adding it later via Flyway alter is clean; removing it after it ships is painful if any code references it prematurely.
Admin frontend (Client Admin + Platform Admin) has no prompt coverage	M	H	BRD Section 24 defines 3 Client Admin UIs and 2 Platform Admin UIs. Prompt Doc only covers partner-facing pages. Client Admins cannot approve redemptions, configure catalogs, or review returns without these UIs. Must be scoped into Sprint 1 frontend work even if not prompted yet.
XTRM/Xoxoday sandbox credentials not confirmed as available	M	H	Adapter development (Sprint 1) requires sandbox API keys. No confirmation in either document that sandbox access has been provisioned. Unblock this in next 48h — adapter code is useless without a testable endpoint.
Notification service integration point undefined	M	M	Eleven notification events are defined in BRD Section 19, but no prompt generates the notification publisher calls. If the notification service is owned by another team, the interface contract (event topic, payload schema) must be confirmed before any service method can publish.

9. Recommended Next Actions (Next 48–72 Hours)

Execute in sequence. Each action has a clear owner and a definition of done.

Action 1 (0–4 hrs): Remove EXPIRY_DEBIT from V002 Migration Draft

Owner: Lead backend engineer.

Do: Edit Prompt Doc Section 3.2 to remove EXPIRY_DEBIT from the LedgerEntry entryType enum. Confirm that the RedemptionOrchestrationService and WalletService prompts contain no reference to it. Done when no Phase 1 artifact mentions EXPIRY_DEBIT.

Action 2 (0–8 hrs): Decide and Document Webhook Dead-Letter Strategy

Owner: Tech Lead + Platform Architect.

Do: Decide between: (a) DB table redemption_webhook_dlq with columns raw_payload, vendor, error_reason, received_at, retry_count, or (b) Kafka DLQ topic tenx.redemption.webhook.dlq. Document the decision in the Prompt Doc as a new Section 4.5 addendum. Create the Flyway migration or Kafka topic config. Done when the dead-letter path is implemented in both XtrmWebhookService and XoxodayWebhookService.

Action 3 (0–8 hrs): Define Kafka Topic Names and Event Schema

Owner: Tech Lead.

Do: Define all 15 topic names from BRD Section 13 using the pattern tenx.redemption.<event_type> and tenx.wallet.<event_type>. Decide schema format (JSON recommended for Sprint 1; Avro for Phase 2). Add a new Prompt Doc Section 4.x — RedemptionEventPublisher — that generates a @Component class with a publish method for each event type. Done when topic names are committed to the platform contracts repo and the prompt is added to the Prompt Doc.

Action 4 (4–16 hrs): Confirm XTRM + Xoxoday Sandbox Access

Owner: Product Manager + Lead Backend Engineer.

Do: Email/Slack both vendor contacts requesting sandbox API keys, base URLs, webhook signing secrets, and test account setup. Store credentials in the team secrets manager (not in code or Prompt Doc). Done when both adapter classes can be invoked against sandbox environments and return non-error responses.

Action 5 (8–24 hrs): Clarify Reward Earning Integration Point

Owner: Tech Lead + Incentives/Engagement POD lead.

Do: Answer: does the existing reward earning system need to be modified in Sprint 1 to call WalletService.credit() when a partner earns a reward? Or are existing TransactionType.REWARD records sufficient for Phase 1 (wallets start at zero until a new earning event triggers credit())? Document the answer as a comment in the BRD Open Questions table (Section 22). Done when the earning-to-wallet integration path is unambiguous and both PODs agree.

Action 6 (8–24 hrs): Add Admin Frontend Prompts to Prompt Doc

Owner: Lead Frontend Engineer.

Do: Add three new prompts to Prompt Doc Section 5: (a) ClientAdminApprovalQueue — pending redemption and return request queue with approve/reject actions; (b) ClientAdminCatalogConfigurator — enable/disable items, set processing mode overrides and thresholds; (c) TenantReportingDashboard — transaction history, filters, export, and redemption rate analytics. Done when prompts follow the same pattern as existing Section 5 prompts and cover all acceptance criteria from BRD Section 18.

Action 7 (16–32 hrs): Spec Platform Admin Catalog Backend Endpoints

Owner: Backend Engineer + Tech Lead.

Do: Define the missing platform admin API surface: POST/PUT/DELETE /api/v1/admin/redemption/catalog (manage global catalog items), GET /api/v1/admin/redemption/catalog/sync (trigger Xoxoday sync), GET /api/v1/admin/vendor/health. Add a Section 4.8 prompt to the Prompt Doc for PlatformAdminCatalogController and accompanying service methods. Done when endpoints are spec'd, permission-gated on action.redemption.catalog.manage, and the Prompt Doc reflects them.

Action 8 (24–48 hrs): Run Full Prompt Sequence Steps 1–4 (Data Model) and Review

Owner: Lead Backend Engineer + all backend team members.

Do: Execute Prompt Doc Sections 2 and 3.1–3.3 in sequence to generate all 6 entity classes and 6 Flyway migrations. Review output against the following checklist: (a) No EXPIRY_DEBIT in V002; (b) @Version on RewardWallet for optimistic locking; (c) RedemptionCatalogItem has NO client_id or @Filter; (d) all BigDecimal columns are NUMERIC(19,4); (e) unique constraint on (client_id, catalog_item_id) in ClientRedemptionConfig; (f) amount > 0 CHECK constraint on LedgerEntry. Done when all 6 migrations pass flyway validate in the dev DB and all entity classes compile cleanly.